

Scaling Automatic Research Agents via World Models

Xiyuan Yang^{1†}, Sheikh Sarwar², Jingru Cheng², Zhan Shi², Duanshun Li²,
Huiyuan Chen², Haiyang Zhang², Chenlei Guo², Jingrui He^{1*}, Zhenyu Liao^{2*}

¹University of Illinois Urbana-Champaign ²Amazon *Corresponding authors

Abstract: Automating empirical research is a long-standing direction of AI. Recent automatic research (AutoResearch) agents bring this goal within reach, as modern LLMs show the capability to independently implement solutions and learn from the execution outcomes. Behind these gains, post-training (especially RL) plays a central role. In this paper, we identify a fundamental tension when scaling RL for these agents: the two components of every AutoResearch trajectory (agent generation and environment execution) scale in very different manners, since all generation shares compute through batching, while each execution occupies its exclusive sandbox and real machine time. As a result, the environment execution dominates the training cost and becomes the bottleneck as trajectories grow. To resolve this tension, we propose *World Model RL* (WMRL), which replaces environment execution with a world model to remove this bottleneck. Additionally, the world model can be imperfect, as its rewards are corrupted by bias and noise. Therefore, we further equip WMRL with two mitigations, *Online Debiasing* and *Inverse-Variance Denoising*, which offset the bias and suppress the noise respectively. Theoretically, we prove that both mitigations of WMRL strictly improve the convergence guarantee. Empirically, WMRL accelerates training by 3–4 $\times$ on various tasks at different agent scales, while exceeding the performance of standard RL baselines. Moreover, our post-trained 4B and 9B agents outperform much larger open-weight agents of 48B and 120B on held-out benchmarks. Beyond AutoResearch, WMRL also transfers to post-training embodied VLA policies, which demonstrates the generalizability of our method.

1. Introduction

An Automatic Research (AutoResearch) agent is a language model that independently conducts empirical research [1, 2, 3]. Given a research question, it formulates an idea, implements the experiment, analyzes the outcome, and iterates [4, 5, 6]. Such agents have already proven capable across various domains. In the natural sciences, for example, they design chemical syntheses and propose reaction conditions that are validated by wet-lab experiments later [7, 8, 9]; in machine learning and data science, they explore real

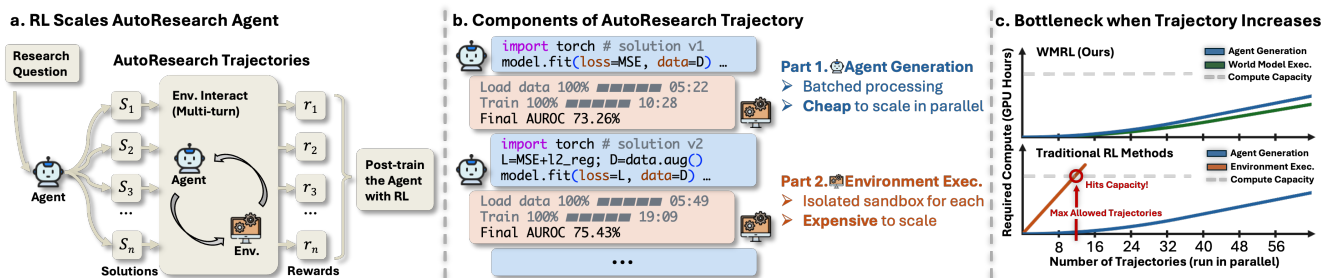


Figure 1: AutoResearch trajectories scale asymmetrically, making environment execution the bottleneck, and our WMRL removes it. (a) Per research question, the agent proposes a group of solutions S_i , graded by execution into rewards r_i , and RL demands a massive number of such trajectories. **(b)** Per trajectory, generation amortizes compute via batching, but execution needs one isolated sandbox per solution. **(c)** Execution compute thus hits capacity first in traditional RL, whereas WMRL scales without this limit.

[†]Work done during an internship at Amazon.

datasets and build training pipelines that outperform human experts [10, 11, 12]. In an AutoResearch task, the agent iteratively generates and executes solutions. These interactions form trajectories, and the execution outcomes provide rewards. With both trajectories and rewards in place, AutoResearch is a natural fit for reinforcement learning (RL) [13, 14, 15, 16], a promising direction to further improve these capabilities.

Like most of RL’s successes, training strong AutoResearch agents demands scale, i.e., massive online trajectories collected during training [16]. However, we identify a fundamental tension underneath this demand: the two components of a trajectory (i.e., agent generation and environment execution) scale in different manners as the trajectory volume grows. On the generation side, rollouts are served by modern inference backends (e.g., vLLM [17] and SGLang [18]), where batching techniques allow concurrent trajectories to share compute, making the cost of additional trajectories negligible. In contrast, execution cost cannot be amortized in this way. Every candidate solution must run in an isolated sandbox that loads the data and trains models on real GPUs [19, 20], so each additional trajectory incurs the full cost, and the total cost grows linearly with the number of trajectories. This asymmetry makes environment execution the dominant bottleneck when scaling RL for AutoResearch agents. Such a bottleneck motivates two research questions:

- (i) *Can we replace the expensive environment (bottleneck) with a fast and scalable signal?*
- (ii) *There is no free lunch, so what does this signal cost? (And how do we pay the bill?)*

The answer to question (i) is to introduce a world model [21, 22], which takes the environment information and the agent solution as input, and produces a simulation of real execution result as output. Such a simulation (though potentially flawed) takes only a few forward passes without real execution, so it batches and amortizes across trajectories just like generation. In RL training, we let the agent interact with the world model instead of the real environment, and receive rewards from the predicted outcomes. As a result, the expensive execution now scales as gracefully as generation and no longer bounds the scale of RL.

We answer question (ii) by modeling the world model’s imperfection as a bias and a noise on the reward, and tracing both through the policy gradient into the convergence bound. Specifically, we consider the world model’s output as an erroneous signal that deviates from real execution by a bias term b with $|b| \leq B$ and a zero-mean noise ξ with standard deviation σ . As shown in Theorem 3, the bias and the noise hinder the final convergence through two extra error terms of $O(B^2)$ and $O(\sigma^2)$ respectively. To reduce both terms, we introduce an *Online Debiasing* mechanism that recasts the world model outputs to offset the bias, and an *Inverse-Variance Denoising* mechanism to minimize the variance; we further ground both mechanisms in Theorem 4 with a strictly improved convergence guarantee.

Our contributions are summarized as follows:

- We introduce the world model into the RL training of AutoResearch agents, which replaces the expensive environment execution with a few forward passes. It removes the critical scaling bottleneck and accelerates overall training by 3–4 $\times$ (Section 3.2).
- We design two correction mechanisms, Online Debiasing and Inverse-Variance Denoising, to counteract the bias and the noise of the world model. With both mechanisms, the accelerated training matches or even exceeds the performance of training with the real environment (Section 3.3).
- We theoretically ground the entire framework. An imperfect world model introduces two error terms into the convergence bound (Section 4.2), and our two mechanisms provably reduce both, yielding a strictly improved convergence guarantee (Section 4.3).

- Through experiments on various AutoResearch tasks across two agent scales, we validate both the efficiency and the performance gains (Section 5.2); we further extend our method to VLA post-training tasks to demonstrate its generalizability (Section 5.3).

2. Related Work

AutoResearch agents and their post-training. Language model agents now carry out substantial parts of empirical research, from autonomous chemical experimentation [7] and open-ended scientific discovery [1] to machine learning engineering, where agents explore the space of solution code [10, 23, 24], curate data as agentic data scientists [25, 26], and pursue recursive self-improvement [27, 28]. A line of benchmarks measures this ability on real Kaggle-style competitions, including MLE-Bench [29], its interactive superset MLE-Dojo [19], DS Bench [30], and MLGym [31], and software engineering agents follow the same execution-driven recipe on real repositories [32, 33, 34], with training environments built at scale [35]. Beyond prompting, RL post-training further improves such agents, typically with group-relative objectives [14] served by large-scale RL systems [36], and synthetic tasks can scale the training corpus [37]. In these pipelines the reward comes from executing the agent’s solution, so training inherits the execution bottleneck of Section 1, the problem we address. Such advances refine how rewards are consumed [38, 39], whereas we change where they come from, so the two are orthogonal and compose.

Learned reward signals and their errors. Replacing an expensive ground truth with a learned signal is a recurring pattern, from reward models in RLHF [40] to LLM judges of model outputs [41], and world models that simulate an environment for control have long powered model-based RL [21, 22, 42]. Such world models are now scaled to foundation models of the physical world [43] or written as executable code by LLMs [44, 45], and learned surrogates of the execution environment have recently been explored for software agents [20]. The errors of such proxies are equally well documented, as optimizing against an imperfect reward model degrades true performance once the proxy is over-trusted [46, 47], and the theory of SGD with biased gradients shows that a systematic gradient error puts a floor on convergence that no amount of training removes [48]. Prior remedies mostly recalibrate the proxy offline [49, 50] or constrain the policy from exploiting it. In off-policy RL, a related line corrects the distribution mismatch between the replay buffer and the target policy with learned correction ratios [51, 52], a finer object than the reward statistics we treat. We instead keep a small stream of ground truth inside the training loop, correct the bias and the variance of the proxy online, and quantify both corrections directly in the convergence bound.

3. Method

In this section, we introduce our WMRL as follows. First, we formalize the standard RL training procedure and define the notation (Section 3.1). Next, we answer the question (i) by incorporating the world model into the RL pipeline as a fast and scalable signal (Section 3.2). We then answer the question (ii) by revealing the additional error terms in the RL convergence, and then providing our mitigations (Section 3.3), which offset these error terms introduced by the world model.

3.1. Preliminaries

Typically, an AutoResearch task provides the agent with the research question, data format and the output requirements. With this context, the agent policy π_θ interacts with an environment (e.g., an isolated Docker

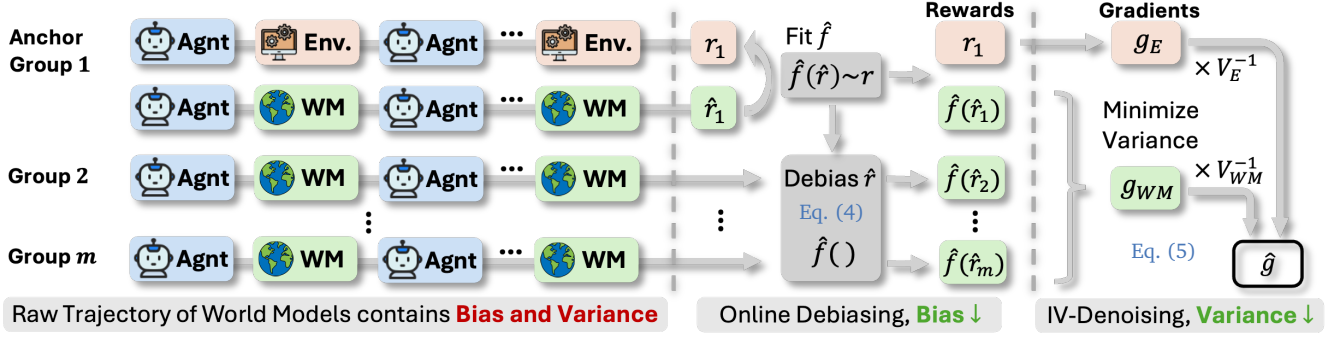


Figure 2: WMRL corrects the world model rewards in two steps. Each row is one of the m groups in a batch. Every group is graded by the world model into biased, noisy scores $\hat{r}$, and anchor groups are also graded by real execution. The monotone map $\hat{f}$ is fit on the score pairs (*Online Debiasing*) to remove the bias, and g_E and g_{WM} are fused by *Inverse-Variance Denoising* to lower the update variance.

container) [19, 30] over multiple turns to iteratively refine its solution until reaching a predefined criterion. Such interactions form a trajectory τ , and the solution receives a score $r(\tau) \in [0, 1]$ that measures its quality.

To enhance the agent policy, a wide range of works leverage RL to maximize the expected score $J(\theta) := \mathbb{E}_{\tau \sim \pi_\theta} [r(\tau)]$ with group-relative policy optimization (GRPO) [14]¹ as follows: For each task, GRPO samples a group of $n \geq 2$ independent trajectories $\tau_1, \dots, \tau_n$ in parallel, and then gives the grade $r(\tau_i)$ of each trajectory by executing the final solution in the environment. The scores are then normalized within the group into advantages $A_i := r(\tau_i) - \frac{1}{n} \sum_j r(\tau_j)$, which yield the gradient estimate as:

$$\hat{g} := \frac{1}{n} \sum_{i=1}^n A_i s_i, \quad s_i := \nabla_\theta \log \pi_\theta(\tau_i), \quad (1)$$

where s_i is the raw policy gradient, and the parameter is updated by $\theta_{t+1} = \theta_t + \gamma \hat{g}_t$ over T steps with step size γ . Throughout training, every step costs online rewards, and each of the rewards requires executing a solution in a new environment with exclusive GPU assignments.

3.2. World Model as an Environment

To provide fast and scalable environment signals (asked in question (i)), we introduce a world model to replace the expensive execution, which constitutes the backbone of WMRL. In our setting, we adopt as the world model a general language model that simulates the environment (with potential errors), instantiated with the same backbone as the agent and queried for the execution outcome. It takes the same task context and an agentic solution as input, and predicts the execution outcome as output, from which an estimated score $\hat{r}(\tau_i)$ is read off in place of the true $r(\tau_i)$. Since this interface is identical to that of the real environment, the pipeline in Section 3.1 runs unchanged with the estimated scores as:

$$\hat{A}_i := \hat{r}(\tau_i) - \frac{1}{n} \sum_j \hat{r}(\tau_j), \quad (2)$$

¹We adopt GRPO as the most common objective for agentic RL at scale [16, 36], while the specific RL objective is orthogonal to our contribution, which acts mostly on the reward side rather than on the update rule.

and the gradient estimate of Equation (1) is computed with $\hat{A}_i$ in place of A_i . In effect, the parameters now ascend the surrogate objective $\mathbb{E}_{\tau \sim \pi_\theta}[\hat{r}(\tau)]$ rather than $J(\theta)$. As a result, the execution part of a trajectory now scales as gracefully as the generation side, which directly removes the execution bottleneck. However, we note that this replacement is not free, as an imperfect predicted outcome can deviate from the real execution outcome. We characterize and mitigate such deviation in Section 3.3 next.

3.3. Anchor Signal for Error Correction

We now answer question (ii). For any world model, the deviation of its predicted score can be decomposed into a systematic part and a random part as follows:

$$\hat{r}(\tau) = r(\tau) + b(\tau) + \xi(\tau), \quad (3)$$

where the bias $b(\tau) := \mathbb{E}[\hat{r}(\tau) \mid \tau] - r(\tau)$ collects the systematic part that survives averaging, and the noise $\xi(\tau)$ is the zero-mean remainder. Since the scores are naturally bounded, we write $B := \sup_\tau |b(\tau)|$ for the maximal magnitude of the bias and $\sigma := \sup_\tau \text{std}(\xi(\tau))$ for the maximal standard deviation of the noise. The remark below summarizes how the two parts affect the convergence.

Remark 1 (The price of the world model replacement). *With real execution rewards, standard RL converges with a bound $J^* - \mathbb{E}[J(\theta_T)] \leq \varepsilon(T)$ on the gap to the optimal score $J^* := \sup_\theta J(\theta)$. With world model rewards, the same analysis gives*

$$J^* - \mathbb{E}[J(\theta_T)] \leq \varepsilon(T) + O(B^2) + O(\sigma^2),$$

where $\varepsilon(T)$ is the standard convergence term (Theorem 3). We next resolve the $O(B^2)$ and $O(\sigma^2)$ terms.

Our goal is to remove or reduce the two additional errors (the $O(B^2)$ and $O(\sigma^2)$ term) in Remark 1. Removing them starts with estimating them, and by Equation (3) both are defined against the true score $r(\tau)$, which the world model itself does not provide. Since the truth can only come from environment execution, we take a small step back and reintroduce a marginal amount of execution for this estimation (which we prove to be enough in Section 4). WMRL therefore keeps a thin stream of ground truth during training, which we call the *anchor signal*. A small fraction of the groups (*anchor groups*) is graded by both the world model and real execution,² and the resulting score pairs $\mathcal{P} = \{(\hat{r}_j, r_j)\}$ feed the two mechanisms below (Figure 2).

We first correct the **bias** by *Online Debiasing*, a monotone distribution recalibration. The training score pairs reveal how the world model score drifts from the truth, so we fit a mapping function over all of them,

$$\hat{f} = \arg \min_f \sum_{(\hat{r}_j, r_j) \in \mathcal{P}} (f(\hat{r}_j) - r_j)^2, \quad (4)$$

where f ranges over monotone functions solved by isotonic regression [49]. With this mapping, all world model scores are then recast into $\hat{f}(\hat{r}(\tau_i))$ before the advantages are formed, and $\hat{f}$ is refit as new pairs arrive on each step to track the drift over training.

For the **noise** part, since it cannot be estimated pointwise, we turn to suppress it by *Inverse-Variance Denoising*, which fuses the two reward streams to reach a lower variance. In each step the m group indices split into the anchor set $\mathcal{G}_E$ and the world model set $\mathcal{G}_{WM}$, and every group k gives an estimate $\hat{g}_k$ of the same policy gradient, with variance V_E or V_{WM} by how it was graded. The question is therefore how to combine them, as

²In practice, anchor groups make up about 10% of all groups.

one stream is scarce but free of world model noise and the other abundant but noisy. Following Lemma 12, the minimal-variance combination weights each stream by the inverse of its variance and attains a variance strictly below either stream alone, so we combine them as

$$\hat{g} = \frac{V_E^{-1}g_E + V_{WM}^{-1}g_{WM}}{V_E^{-1}|\mathcal{G}_E| + V_{WM}^{-1}|\mathcal{G}_{WM}|}, \quad g_E := \sum_{k \in \mathcal{G}_E} \hat{g}_k, \quad g_{WM} := \sum_{k \in \mathcal{G}_{WM}} \hat{g}_k, \quad (5)$$

where the stream gradients g_E and g_{WM} sum the group estimates $\hat{g}_k = \frac{1}{n} \sum_{i=1}^n A_{k,i} s_{k,i}$, each applying Equation (1) to group k with the true scores on $\mathcal{G}_E$ and the calibrated scores on $\mathcal{G}_{WM}$.

Equation (5) is the conceptual form of the fusion, and it attains the minimal variance once V_E and V_{WM} are given. In implementation, Equation (5) reduces to the simple form $\hat{g} = (\rho g_E + g_{WM}) / (\rho |\mathcal{G}_E| + |\mathcal{G}_{WM}|)$, with only a single unknown left, the ratio $\rho := V_{WM} / V_E$. To derive this ratio, we first show in Section 4 that $\rho = 1 + c \sigma^2$, in which the reward noise σ^2 is measurable and the constant c is estimable. We then measure σ^2 by the mean squared residual $\hat{\eta}^2$ between the calibrated and the true scores on the anchor groups, and estimate c by the reciprocal of one such residual recorded at the end of a warmup phase, $c := 1 / \hat{\eta}_{\text{cal}}^2$. The anchor groups then enter Equation (5) with the weight $\rho = 1 + \hat{\eta}^2 / \hat{\eta}_{\text{cal}}^2$ and the world model groups with the weight one, a rule that carries no free parameter. We further justify the optimality of this estimate both theoretically and empirically in Remark 8, as the leading-order term of the estimation error vanishes exactly by construction and the remaining higher-order excess measures a few percent at most in our runs. Both mitigations are validated empirically in Section 5 and analyzed in Section 4.

3.4. Discussion

We note that the proposed denoising weights are not only theoretically sound (Section 4) but also empirically interpretable. On the anchor groups, the tracked residual $\hat{\eta}^2$ measures how far the calibrated predictions fall from the true scores, and thereby audits the world model throughout training. When the audit worsens, the anchor weight in Equation (5) rises. Consider a task whose outcome hinges on randomness that no reading of the solution reveals. In this case, $\hat{\eta}^2$ remains large and the gradient is carried by the anchor groups, so WMRL degrades toward the standard GRPO of Section 3.1 instead of learning from a corrupted signal. This fallback follows from the measurement itself rather than a mixing ratio fixed in advance.

Additionally, although the design targets the execution bottleneck of AutoResearch, it is not limited to this scenario. The construction requires only that rewards are expensive to execute yet predictable from the artifacts the agent produces, and that a small stream of ground truth stays available for anchoring. Post-training embodied policies is one such instance, where real rollouts are slow while learned simulators are cheap, and we validate this transfer on VLA tasks in Section 5.

4. Theoretical Analysis

In this section, we first analyze the additional error terms that an imperfect world model introduces, and then justify our mitigation by a convergence analysis. We first bound the convergence of training on world model rewards alone in Section 4.2 (Theorem 3), and then bound the convergence of WMRL in Section 4.3 (Theorem 4), and compare the two term by term. We put the full proofs and auxiliary lemmas in Appendix B.

4.1. Setup

We analyze the RL training procedure of Section 3.1, namely T ascent steps with step size γ on the gradient estimator of Equation (1), and we measure progress by the expected gap to the optimal score, $J^* - \mathbb{E}[J(\theta_T)]$ with $J^* := \sup_{\theta} J(\theta)$ and $\Delta_0 := J^* - J(\theta_0)$. The analysis rests on one standard assumption.

Assumption 2 (Regularity). J is L -smooth and satisfies the gradient domination condition $\|\nabla J(\theta)\|^2 \geq 2\mu (J^* - J(\theta))$ for some $\mu > 0$, and the log-likelihood gradient is bounded by $\|\nabla_{\theta} \log \pi_{\theta}(\tau)\| \leq M$ for all τ .

Everything the world model contributes enters through the two quantities of Section 3.3, the bias magnitude B and the noise standard deviation σ of Equation (3), and through the two group variances V_E and V_{WM} of Equation (5). The gradient estimator of Equation (1) is linear in the rewards, so the two error parts act on the gradient separately. The bias is deterministic and shifts its mean. The noise is zero-mean and only inflates the variance of a world model group, to $V_{WM} = (1 + c\sigma^2) V_E$ with $c = 4M^2/(nV_E)$, as announced in Section 3.3. Only the product $c\sigma^2$ enters the bounds, and Section 3.3 supplies it by measuring its varying part and normalizing its scale, so neither is needed on its own. Throughout we take $\gamma \leq 1/(8L)$.

4.2. The Cost of World Model Rewards

We first consider the setting of Section 3.2, where every group is graded by the world model and no real execution takes place, so the rewards carry the bias and the noise of Equation (3) at every step. Since the gradient estimator built on Equation (2) is linear in the rewards, the two deviations reach the gradient in different ways, the bias shifting its mean and the noise inflating its variance, and they therefore enter the convergence bound as the two separate terms announced in Remark 1.

Theorem 3 (Convergence with world model rewards). *Under Assumption 2, GRPO trained on world model rewards satisfies*

$$J^* - \mathbb{E}[J(\theta_T)] \leq \left(1 - \frac{\gamma\mu}{4}\right)^T \Delta_0 + \underbrace{O(M^2 B^2)}_{\text{world model bias}} + \underbrace{O(\gamma V_{WM})}_{\text{world model variance}},$$

where $V_{WM} = (1 + c\sigma^2) V_E$ carries the noise, and the variance term is presented as $O(\sigma^2)$ in Remark 1, and the hidden constants depend only on L and μ . We put the full proof in Appendix B.2.

The first term is the standard geometric decay of the noise-free RL setting and vanishes as training proceeds. The bias term contains neither T nor γ , hence no amount of training and no choice of step size removes it, and the achievable score stays capped by the bias of the world model. The variance term grows with the noise through V_{WM} and likewise inflates the final error, so both terms call for correction.

4.3. The Effect of Our Mitigation

We now analyze WMRL, whose two corrections from Section 3.3 improve one term of Theorem 3 each. Both corrections rely on the anchor signal, where real execution grades a small fraction of the groups and yields score pairs throughout training. With these pairs, the Online Debiasing of Equation (4) keeps shrinking the residual bias, at a speed captured by a constant T_0 given in Appendix B.3. The Inverse-Variance Denoising of Equation (5) then fuses the two gradient streams, so every update carries less noise than either stream alone. In the statement below, $\tilde{O}$ hides logarithmic factors.

Theorem 4 (Convergence of WMRL). *Under Assumption 2 and the conditions of Appendix B.3, WMRL satisfies with high probability*

$$J^* - \mathbb{E}[J(\theta_T)] \leq \left(1 - \frac{\gamma\mu}{4}\right)^T \Delta_0 + \underbrace{\tilde{O}\left(\frac{M^2 B^2}{1 + T/T_0}\right)}_{\text{contractive bias}} + \underbrace{O\left(\frac{\gamma V_{WM}}{1 + V_{WM}/V_E}\right)}_{\text{reduced variance}}.$$

We put the full proof in Appendix B.3.

The two bounds share the same leading term and align term by term, with each remaining term of Theorem 4 equal to its counterpart in Theorem 3 divided by a factor larger than one. The bias term is the same $M^2 B^2$ divided by $1 + T/T_0$, so the permanent floor contracts as training proceeds and vanishes in the limit. The variance term is the same γV_{WM} divided by $1 + V_{WM}/V_E$, which lands below what either stream attains alone, and the factor grows as the anchor stream becomes comparatively more reliable. Both terms are therefore strictly smaller once the world model is biased or noisy at all, and letting $T \rightarrow \infty$ removes the bias term entirely. In this sense WMRL converges to the same optimum as training with real execution, while paying for real execution on a small fraction of the groups.

5. Experiments

We empirically validate the effectiveness of WMRL on the two questions in Section 1, i.e., whether the world model delivers the promised speedup (question (i)) and whether it costs any final performance (question (ii)), with the main comparison in Section 5.2 (Table 1). We then show how our recipe generalizes to other post-training domains in Section 5.3 (Table 2) and how the two corrections act in Section 5.4 (Table 3).

5.1. Experimental Setup

Benchmarks. We evaluate WMRL on various domains, including AutoResearch tasks and vision-language-action (VLA) post-training tasks. In the AutoResearch evaluation, we use MLE-Dojo [19], an interactive superset of MLE-Bench [29] built on Kaggle machine learning competitions, and follow the common practice [53, 54, 55] of manually dividing the pool into MLE-Dojo (train) and MLE-Dojo (test) in a category-balanced way, since the original test split of MLE-Bench has several evaluation issues (Appendix C). We also put the selection rule and the full task lists in Appendix C. Beyond the MLE-style tasks, we further evaluate on DSbench [30], a data science benchmark also built on Kaggle modeling tasks and fully disjoint from our training set. In the VLA evaluation, we use LIBERO-Long [56], a standard simulated manipulation benchmark, where the policy controls a robot arm to complete long-horizon tasks from image observations and language instructions.

Models and training. For the AutoResearch tasks, we post-train research agents at two scales, Qwen3.5-4B and Qwen3.5-9B [57], with the RL procedure of Section 3. For the world model, we use the same backbone as the agent at each scale and prompt it for outcome prediction (Appendix D). Sharing one backbone also rules out any external knowledge, so the gains cannot come from implicitly distilling a stronger model. Throughout training, only the research agent is updated while the world model stays unchanged. All the runs use identical A100 GPU allocations, and we report the total training compute in GPU-hours. For the VLA tasks, the agent is MiniVLA-1B [58], which is a compact vision-language-action model pretrained on LIBERO-90, and the world model is Robometer [59], an off-the-shelf VLM that predicts task success rate

Table 1: **Main results (leaderboard percentile in %, higher is better)**. Both benchmarks hold out tasks never trained on, and **GPU-hours** count the total training compute. Scores are per-task avg@8, averaged within each category, and **Avg** is the mean over categories. **Bold** marks the best value per column, and green (red) arrows mark the gain (drop) of WMRL over same-scale GRPO.

Method	GPU-hours	MLE-Dojo (test)				DSBench				
		Tab	Text	Img	Avg	Bin-Cls	Multi-Cls	Regress	Other	Avg
Baseline Models										
Qwen3.5-4B	—	13.7	8.0	0.2	7.3	21.9	11.8	25.8	8.8	17.1
Qwen3.5-9B	—	17.4	8.1	3.4	9.6	29.1	16.0	36.3	14.1	23.9
Large Agents										
Kimi-48B-A3B	—	12.3	10.1	2.0	8.1	20.3	10.0	29.0	9.8	17.3
Nemotron-120B-A12B	—	35.1	18.8	7.6	20.5	38.7	26.6	40.7	20.6	31.7
RL in Real Environment										
Qwen3.5-4B-GRPO	883	24.8	16.6	4.3	15.2	34.3	17.8	28.3	22.4	25.7
Qwen3.5-9B-GRPO	1174	33.3	17.1	5.9	18.8	40.9	26.8	40.4	16.7	31.2
RL with Pure World Model										
Qwen3.5-4B-WM	269	21.2	14.6	3.0	12.9	27.1	16.9	32.5	16.0	23.1
Qwen3.5-9B-WM	330	28.0	15.9	4.5	16.1	35.4	21.8	37.9	16.9	28.0
WMRL (ours)										
Qwen3.5-4B-Ours	286	28.0↑3.2	16.1↓0.5	5.2↑0.9	16.4↑1.2	35.8↑1.5	22.3↑4.5	32.3↑4.0	24.7↑2.3	28.8↑3.1
Qwen3.5-9B-Ours	349	38.0↑4.7	19.4↑2.3	7.4↑1.5	21.6↑2.8	45.4↑4.5	26.9↑0.1	39.2↓1.2	19.8↑3.1	32.8↑1.6

from images. We first finetune the agent on the official demonstrations of each task, and then post-train it with the same RL procedure. The remaining details and hyperparameters are listed in Appendix D.

Metrics. For the AutoResearch tasks, we adopt the official metric of MLE-Dojo [19, 54], which scores each submission by its percentile on the corresponding real competition leaderboard. The percentile is naturally normalized (within 0 – 1), so scores are comparable across tasks. For the VLA tasks, we score each rollout by whether it succeeds. In both domains, every result we report is an average over eight attempts (avg@8).

Baselines. We compare WMRL with four types of baselines. The untrained bases Qwen3.5-4B and Qwen3.5-9B give the starting level of the backbone before any post-training. The large open-weight agents Kimi-48B-A3B and Nemotron-120B-A12B serve as strong off-the-shelf references and test whether model scale alone can replace post-training. GRPO in the real environment is the standard full-cost training that WMRL aims to match, and is our primary comparison. The pure world model baseline trains on predicted rewards alone and shows what the raw world model delivers without any correction. All trained models share the scaffold, the hyperparameters, and the data split.

5.2. Main Results on AutoResearch Tasks

Table 1 compares WMRL with the four types of baselines of Section 5.1, at both scales and on both benchmarks, and examines whether WMRL delivers the promised speedup and whether the cheap signal costs any final performance. The answer is positive on both counts. First, WMRL cuts the training compute of real-execution GRPO by $3.1\times$ and $3.4\times$ and still scores higher on every benchmark, with gains of up to 3.1 points, so the saving arrives at no cost in final performance. Second, the post-trained agents beat far larger off-the-shelf

Table 2: **VLA post-training results on LIBERO-Long (success rate in %, higher is better)**. We report in-domain performance, out-of-distribution generalization, and the overall average, where Overall averages over every initial state, seen and unseen. Best@8 counts at least one success of the eight and All@8 counts all eight. All RL rows share the SFT initialization. **Bold** marks the best value per column, and green (red) arrows mark the gain (drop) of WMRL over MiniVLA-1B-GRPO.

Method	In-Domain			Out-of-Distribution			Overall
	Avg	Best@8	All@8	Avg	Best@8	All@8	
Baseline Models							
MiniVLA-1B	5.6	5.6	5.6	2.9	2.9	2.9	3.8
MiniVLA-1B-SFT	37.3	41.3	33.1	37.5	44.7	32.1	37.4
RL in Real Environment							
MiniVLA-1B-GRPO	39.3	48.8	34.4	37.8	45.3	31.2	38.3
RL with Pure World Model							
MiniVLA-1B-WM	39.1	45.6	34.4	39.3	45.6	34.1	39.2
WMRL (ours)							
MiniVLA-1B-Ours	41.2 $\uparrow 1.9$	47.5 $\downarrow 1.3$	37.5 $\uparrow 3.1$	41.2 $\uparrow 3.4$	48.8 $\uparrow 3.5$	37.1 $\uparrow 5.9$	41.2 $\uparrow 2.9$

agents, as our 4B model surpasses the 48B agent and our 9B model surpasses the 120B agent on both averages.

5.3. Generalization Ability on Embodied VLAs

We next test whether WMRL transfers to other domains, and we pick vision-language-action (VLA) post-training, an active line of embodied learning [60, 61, 62]. The two calibration mechanisms are not specific to AutoResearch, so we apply the same recipe to the VLA setup of Section 5.1. Here Robometer predicts a progress value for eight frames sampled from each rollout as the dense reward, and the single sparse success that the environment returns at the end of a rollout serves as the anchor. The resulting success rates in Table 2 repeat the pattern of Table 1. Either signal alone barely moves the policy, as RL on the sparse outcome adds 0.9 points over the SFT baseline and RL on the raw world model signal adds 1.8. WMRL combines the two and lifts the overall success rate by 3.8 points, with the largest margin on unseen initial states. The recipe therefore carries over across domains, agent architectures, and reward types.

5.4. Ablation Study

We then ablate the two corrections at both scales (Table 3). All runs consume the same two reward streams at the same ratio as WMRL and differ only in which correction is active. In particular, the first row is the direct mixture that feeds both signals to GRPO with no correction at all. From this baseline we add Inverse-Variance Denoising alone, Online De-biasing alone, and finally both, which

Table 3: **Ablation on the two corrections (leaderboard percentile in %, Avg over categories)**. OD is Online Debiasing, IVD is Inverse-Variance Denoising, a filled circle marks the correction as active, and the last row recovers WMRL. Green arrows give its gain over the uncorrected first row.

OD	IVD	4B Agent		9B Agent	
		MLE	DS	MLE	DS
○	○	13.5	25.3	16.8	29.5
○	●	14.9	26.2	18.0	31.2
●	○	15.7	28.1	19.4	31.7
●	●	16.4 $\uparrow 2.9$	28.8 $\uparrow 3.5$	21.6 $\uparrow 4.8$	32.8 $\uparrow 3.3$

recovers WMRL. The plain mixture is not enough, as its scores stay below real-execution GRPO (Table 1) on every column. Each correction then helps on its own. Inverse-Variance Denoising alone adds 0.9 to 1.7 points, Online Debiasing alone adds 2.2 to 2.8 points, and the larger share of the debiasing gain matches Theorem 3, where the bias enters the bound at full size while the noise enters damped by the step size. Activating both lifts every column by 2.9 to 4.8 points, more than either correction alone, so the two mechanisms are complementary rather than redundant, as each removes the term the other leaves behind.

6. Conclusion

This work scales RL for AutoResearch agents by replacing the expensive environment execution with a world model and correcting its bias and noise through a small anchored stream of real execution. The two corrections turn the permanent error floor of world model training into a contracting term and reduce the variance below either reward stream alone, and they cut the training compute by three to four times while matching or exceeding full real-execution RL at two scales. The transfer to VLA post-training further suggests a general path for scaling RL wherever execution, not generation, is the bottleneck.

References

- [1] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- [2] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, et al. Towards an AI co-scientist. *arXiv preprint arXiv:2502.18864*, 2025.
- [3] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using LLM agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025.
- [4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- [5] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023.
- [6] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- [7] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.
- [8] Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024.
- [9] Nathan J Szymanski, Bernardus Rendy, Yuxing Fei, Rishi E Kumar, Tanjin He, David Milsted, Matthew J McDermott, Max Gallant, Ekin Dogus Cubuk, Amil Merchant, et al. An autonomous laboratory for the accelerated synthesis of novel inorganic materials. *Nature*, 624:86–91, 2023.
- [10] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.
- [11] Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. DS-Agent: Automated data science by empowering large language models with case-based reasoning. In *International Conference on Machine Learning*, 2024.
- [12] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MAgentBench: Evaluating language agents on machine learning experimentation. In *International Conference on Machine Learning*, 2024.
- [13] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [14] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

- [15] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- [16] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [18] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024.
- [19] Rushi Qiang, Yuchen Zhuang, Yinghao Li, Dingu Sagar V K, et al. MLE-Dojo: Interactive environments for empowering LLM agents in machine learning engineering. *arXiv preprint arXiv:2505.07782*, 2025.
- [20] Shuang Sun, Huatong Song, Lisheng Huang, Jinhao Jiang, Ran Le, Zhihao Lv, Zongchao Chen, Yiwen Hu, Wenyang Luo, Wayne Xin Zhao, Yang Song, Hongteng Xu, Tao Zhang, and Ji-Rong Wen. SWE-World: Building software engineering agents in docker-free environments. *arXiv preprint arXiv:2602.03419*, 2026.
- [21] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- [22] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025.
- [23] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [24] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.
- [25] Ilia Kulikov, Chenxi Whitehouse, Tianhao Wu, Yixin Nie, Swarnadeep Saha, Eryk Helenowski, Weizhe Yuan, Olga Golovneva, Jack Lanchantin, Yoram Bachrach, Jakob Foerster, Xian Li, Han Fang, Sainbayar Sukhbaatar, and Jason Weston. Autodata: An agentic data scientist to create high quality synthetic data. *arXiv preprint arXiv:2606.25996*, 2026.
- [26] Martin Bertran, Riccardo Fogliato, and Zhiwei Steven Wu. Many ai analysts, one dataset: Navigating the agentic data science multiverse. *Proceedings of the National Academy of Sciences*, 123(29):e2606495123, 2026.
- [27] Junlin Yang, Che Jiang, Yu Fu, Tianwei Luo, Can Ren, Weizhi Wang, Kaikai Zhao, Hongyi Liu, et al. Frontis-MA1: Training an AI4AI model towards recursive self-improvement in machine learning engineering. *arXiv preprint arXiv:2607.28568*, 2026.
- [28] Recursive. First steps toward automated AI research. <https://www.recursive.com/articles/first-steps-toward-automated-ai-research>, 2026.

- [29] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024.
- [30] Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, et al. DSbench: How far are data science agents from becoming data science experts? *arXiv preprint arXiv:2409.07703*, 2024.
- [31] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, et al. MLGym: A new framework and benchmark for advancing AI research agents. *arXiv preprint arXiv:2502.14499*, 2025.
- [32] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- [33] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- [34] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, 2025.
- [35] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. In *International Conference on Machine Learning*, 2025.
- [36] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, et al. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint arXiv:2409.19256*, 2024.
- [37] Jian Xie, Tianhe Lin, Zilu Wang, Yuting Ning, Yuekun Yao, Tianci Xue, Zhehao Zhang, Zhongyang Li, Kai Zhang, Yufan Wu, Shijie Chen, Boyu Gou, Mingzhe Han, Yifei Wang, Vint Lee, Xinpeng Wei, Xiangjun Wang, Yu Su, and Huan Sun. QUEST: Training frontier deep research agents with fully synthetic tasks. *arXiv preprint arXiv:2605.24218*, 2026.
- [38] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [39] Yuxiao Qu, Amrith Setlur, Virginia Smith, Ruslan Salakhutdinov, and Aviral Kumar. POPE: Learning to reason on hard problems via privileged on-policy exploration. *arXiv preprint arXiv:2601.18779*, 2026.
- [40] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.
- [41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems 36: Datasets and Benchmarks Track*, volume 36, pages 46595–46623, 2023.

- [42] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *International Conference on Machine Learning*, 2024.
- [43] NVIDIA. Cosmos world foundation model platform for physical AI. *arXiv preprint arXiv:2501.03575*, 2025.
- [44] Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by monte carlo tree search. In *Advances in Neural Information Processing Systems*, 2024.
- [45] Hao Tang, Darren Key, and Kevin Ellis. Worldcoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems*, 2024.
- [46] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 10835–10866, 2023.
- [47] Thomas Coste, Usman Anwar, Robert Kirk, and David Krueger. Reward model ensembles help mitigate overoptimization. In *International Conference on Learning Representations*, 2024.
- [48] Ahmad Ajallooeian and Sebastian U. Stich. On the convergence of SGD with biased gradients. *arXiv preprint arXiv:2008.00051*, 2020.
- [49] Bianca Zadrozny and Charles Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2002.
- [50] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1321–1330, 2017.
- [51] Ofir Nachum, Yinlam Chow, Bo Dai, and Lihong Li. DualDICE: Behavior-agnostic estimation of discounted stationary distribution corrections. In *Advances in Neural Information Processing Systems*, 2019.
- [52] Jiachen Li, Shuo Cheng, Zhenyu Liao, Huayan Wang, William Yang Wang, and Qinxun Bai. Off-policy reinforcement learning with optimistic exploration and distribution correction. In *Deep Reinforcement Learning Workshop, NeurIPS*, 2022.
- [53] Zexi Liu, Jingyi Chai, Xinyu Zhu, Shuo Tang, Rui Ye, Bo Zhang, Lei Bai, and Siheng Chen. ML-Agent: Reinforcing LLM agents for autonomous machine learning engineering. *arXiv preprint arXiv:2505.23723*, 2025.
- [54] Yuzhu Cai, Zexi Liu, Xinyu Zhu, Cheng Wang, Yanfeng Wang, and Siheng Chen. AceGRPO: Adaptive curriculum enhanced group relative policy optimization for autonomous machine learning engineering. *arXiv preprint arXiv:2602.07906*, 2026.
- [55] Yuhang Zhou, Lizhu Zhang, Yifan Wu, Jiayi Liu, Xiangjun Fan, Zhuokai Zhao, and Hong Yan. Synthetic sandbox for training machine learning engineering agents. *arXiv preprint arXiv:2604.04872*, 2026.

- [56] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, 2023.
- [57] Qwen Team. Qwen3.5. <https://qwen.ai/blog?id=qwen3.5>, 2026.
- [58] Suneel Belkhale and Dorsa Sadigh. MiniVLA: A better VLA with a smaller footprint. <https://github.com/Stanford-ILIAD/openvla-mini>, 2024.
- [59] Anthony Liang, Yigit Korkmaz, Jiahui Zhang, Minyoung Hwang, Abrar Anwar, Sidhant Kaushik, Aditya Shah, Alex S. Huang, Luke Zettlemoyer, Dieter Fox, Yu Xiang, Anqi Li, Andreea Bobu, Abhishek Gupta, Stephen Tu, Erdem Biyik, and Jesse Zhang. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *arXiv preprint arXiv:2603.02115*, 2026.
- [60] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, 2024.
- [61] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [62] Shuanghao Bai, Jing Lyu, Wanqi Zhou, Zhe Li, Dakai Wang, Lei Xing, Xiaoguang Zhao, Pengwei Wang, Zhongyuan Wang, Cheng Chi, Badong Chen, and Shanghang Zhang. Latent reasoning VLA: Latent thinking and prediction for vision-language-action models. In *International Conference on Machine Learning*, 2026.
- [63] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47:235–256, 2002.
- [64] Jinglin Chen and Nan Jiang. Information-theoretic considerations in batch reinforcement learning. In *International Conference on Machine Learning*, 2019.
- [65] Cun-Hui Zhang. Risk bounds in isotonic regression. *The Annals of Statistics*, 30(2):528–555, 2002.
- [66] OpenAI. MLE-bench: Evaluating machine learning agents on machine learning engineering. <https://openai.com/index/mle-bench/>, 2024.

Appendix

A	Notation	18
B	Proofs	18
B.1	Warm-up: convergence of standard RL	18
B.2	Proof of Theorem 3	19
B.3	Proof of Theorem 4 and the comparison	20
B.4	Auxiliary lemmas	21
C	Choice of the Task Pool	25
D	Experimental Details	27
E	Prompts	29

A. Notation

Table 4 collects the notation of the main text, in order of first appearance.

Table 4: Notation used in the main text.

Symbol	Meaning
$\tau, r(\tau)$	a trajectory and its true score from real execution, $r(\tau) \in [0, 1]$
$\pi_\theta, J(\theta)$	the agent policy and its expected score $\mathbb{E}_{\tau \sim \pi_\theta}[r(\tau)]$
n, A_i, s_i	group size, the advantage, and the policy gradient of trajectory τ_i
$\hat{g}, \gamma, T$	the gradient estimate, the step size, and the number of training steps
$\hat{r}(\tau), \hat{A}_i$	the world model score and the advantage computed from it
$b(\tau), \zeta(\tau)$	the bias and the zero-mean noise of the world model score
B, σ	the maximal bias magnitude and the maximal noise standard deviation
$\mathcal{P}$	the pool of score pairs $(\hat{r}_j, r_j)$ collected on anchor groups
$\hat{f}$	the monotone recalibration map fit on $\mathcal{P}$
$m, \mathcal{G}_E, \mathcal{G}_{WM}$	the groups per step and the index sets of the anchor and world model ones
g_E, g_{WM}	the sums of the group estimates $\hat{g}_k$ over $\mathcal{G}_E$ and over $\mathcal{G}_{WM}$
V_E, V_{WM}	the per-group gradient variances of the two streams
ρ, c	the variance ratio $V_{WM}/V_E = 1 + c\sigma^2$ and its conversion constant
$\hat{\eta}^2, \hat{\eta}_{\text{cal}}^2$	the tracked residual of the calibrated scores and its warmup value
J^*, Δ_0	the optimal score and the initial gap $J^* - J(\theta_0)$
L, μ, M	the smoothness, gradient domination, and gradient bound constants
$\varepsilon(T)$	the convergence term of standard RL (Proposition 5)
T_0	the recalibration time scale of Theorem 4

B. Proofs

This appendix proves every claim of Section 4, in the order of the story. Appendix B.1 derives the standard convergence term $\varepsilon(T)$ of Remark 1 for training with real execution. Appendix B.2 proves Theorem 3 for training on world model rewards, and Appendix B.3 proves Theorem 4 for WMRL and concludes with the term-by-term comparison. The auxiliary lemmas invoked along the way are only cited there, and their statements with full proofs are collected in Appendix B.4. Throughout, the variance of a random vector X is the scalar $\text{Var}(X) := \mathbb{E}\|X - \mathbb{E}X\|^2$, the trace of its covariance matrix, which satisfies $\mathbb{E}\|X\|^2 = \|\mathbb{E}X\|^2 + \text{Var}(X)$.

B.1. Warm-up: convergence of standard RL

We first derive the bound that training with real execution satisfies. It contains no contribution from the world model and is exactly the $\varepsilon(T)$ of Remark 1.

Proposition 5 (Standard RL). *Under Assumption 2 and $\gamma \leq 1/(8L)$, GRPO trained on real execution rewards satisfies*

$$J^* - \mathbb{E}[J(\theta_T)] \leq \left(1 - \frac{\gamma\mu}{4}\right)^T \Delta_0 + \frac{4L\gamma}{\mu} V_E =: \varepsilon(T).$$

Proof. We proceed in three steps.

Step 1, the estimate is clean. Let $\mathbb{E}_t$ denote expectation conditioned on θ_t , so that $J(\theta_t)$ and its gradient are fixed under $\mathbb{E}_t$. By Lemma 10 (i), proved in Appendix B.4, the estimator of Equation (1) satisfies $\mathbb{E}_t[\hat{g}_t] = \kappa \nabla J(\theta_t)$ with $\kappa = 1 - \frac{1}{n} \in [\frac{1}{2}, 1)$ and $\text{Var}(\hat{g}_t | \theta_t) \leq V_E$. Since $\|\mathbb{E}_t \hat{g}_t\| = \kappa \|\nabla J(\theta_t)\| \leq \|\nabla J(\theta_t)\|$, the second moment obeys $\mathbb{E}_t \|\hat{g}_t\|^2 = \|\mathbb{E}_t \hat{g}_t\|^2 + \text{Var}(\hat{g}_t | \theta_t) \leq \|\nabla J(\theta_t)\|^2 + V_E$.

Step 2, one-step progress. Write $\nabla := \nabla J(\theta_t)$. By L -smoothness of J ,

$$J(\theta_{t+1}) \geq J(\theta_t) + \gamma \langle \nabla, \hat{g}_t \rangle - \frac{L\gamma^2}{2} \|\hat{g}_t\|^2.$$

Taking expectations conditioned on θ_t and inserting Step 1,

$$\mathbb{E}_t J(\theta_{t+1}) \geq J(\theta_t) + \gamma \left(\kappa - \frac{L\gamma}{2} \right) \|\nabla\|^2 - \frac{L\gamma^2}{2} V_E \geq J(\theta_t) + \frac{\gamma}{8} \|\nabla\|^2 - \frac{L\gamma^2}{2} V_E,$$

where the last step uses $\kappa \geq \frac{1}{2}$ and $L\gamma \leq \frac{1}{8}$.

Step 3, gradient domination and unrolling. By Assumption 2, $\|\nabla\|^2 \geq 2\mu(J^* - J(\theta_t))$. Writing $\delta_t := J^* - \mathbb{E}[J(\theta_t)]$ and taking total expectations,

$$\delta_{t+1} \leq \left(1 - \frac{\gamma\mu}{4}\right) \delta_t + \frac{L\gamma^2}{2} V_E.$$

Unrolling over $t = 0, \dots, T-1$ and bounding the geometric sum by $\sum_{k \geq 0} (1 - \frac{\gamma\mu}{4})^k = \frac{4}{\gamma\mu}$ gives $\delta_T \leq (1 - \frac{\gamma\mu}{4})^T \Delta_0 + \frac{2L\gamma}{\mu} V_E$, and we state the looser constant $\frac{4L\gamma}{\mu}$ to share constants with the perturbed case below. $\square$

B.2. Proof of Theorem 3

With predicted rewards the estimate is no longer clean. By Lemma 10, stated and proved in Appendix B.4, the estimator computed with predicted scores decomposes as $\hat{g}_k^{WM} = \hat{g}_k + g_b + g_\xi$, where g_b is a bounded deterministic shift induced by the bias and g_ξ is a zero-mean fluctuation induced by the noise. The recursion of Proposition 5 then goes through with two modifications, the variance grows and a perturbation term appears, which is the content of Lemma 9 in Appendix B.4.

Theorem (Convergence with world model rewards, Theorem 3 of Section 4.2 with explicit constants). *Under Assumption 2, GRPO trained on world model rewards with $\gamma \leq 1/(8L)$ satisfies*

$$J^* - \mathbb{E}[J(\theta_T)] \leq \left(1 - \frac{\gamma\mu}{4}\right)^T \Delta_0 + \frac{32M^2}{\mu} B^2 + \frac{4L\gamma}{\mu} V_{WM}.$$

Proof. The plan is to verify the hypotheses of Lemma 9 for the decomposition supplied by Lemma 10, and then read off the bound. By Lemma 10, the estimate computed with predicted scores splits as $u + d$ with $u := \hat{g}_k + g_\xi$ and $d := g_b$. The part u is the clean estimate plus a conditionally zero-mean fluctuation, so its mean is the unchanged $(1 - \frac{1}{n})\nabla J$, and since $\hat{g}_k$ and g_ξ are uncorrelated their variances add, $V_E + 4M^2\sigma^2/n = V_{WM}$. The part d obeys $\|d\| \leq 2MB$ almost surely, which serves as the constant perturbation bound D . Applying Lemma 9 with $\kappa_t = 1 - \frac{1}{n} \geq \frac{1}{2}$, $V = V_{WM}$ and $D = 2MB$ then yields the three terms of the claim, the geometric decay, the constant bias term $\frac{8}{\mu}(2MB)^2 = \frac{32M^2}{\mu} B^2$, and the variance term $\frac{4L\gamma}{\mu} V_{WM}$. $\square$

B.3. Proof of Theorem 4 and the comparison

Compared with Appendix B.2, two things improve for WMRL, the recalibration makes the perturbation bound shrink over time, and the fusion makes the variance harmonic. Analyzing the recalibration needs one modeling assumption beyond Assumption 2.

Assumption 6 (Score scale). *The bias acts on the score scale as a monotone distortion, that is, $b(\tau) = \phi(r(\tau)) - r(\tau)$ for a non-decreasing ϕ , and the noise ξ is independent across trajectories with $|\xi| \leq 1$.*

This is what makes the bias learnable by a monotone fit. How fast it is learned is quantified by Lemma 11, whose constant c_f measures how quickly the recalibration error decays. The constant T_0 of Theorem 4 is $T_0 := c_f^2/B^2$, the number of steps after which the residual bias of the recalibrated scores falls below the raw bias of the world model.

Theorem (Convergence of WMRL, Theorem 4 of Section 4.3 with explicit constants). *Under Assumptions 2 and 6 and $\gamma \leq 1/(8L)$, WMRL satisfies with probability at least $1 - \delta$*

$$\begin{aligned} J^* - \mathbb{E}[J(\theta_T)] \leq & \left(1 - \frac{\gamma\mu}{4}\right)^T \Delta_0 + \frac{64c_f^2M^2}{\mu} \cdot \frac{\log(2KT/\delta)}{T} \\ & + 4\gamma TM^2B^2 \left(1 - \frac{\gamma\mu}{4}\right)^{T/2} + \frac{4L\gamma}{\mu} \left(\frac{1}{V_E} + \frac{1}{V_{WM}}\right)^{-1}, \end{aligned}$$

where V_{WM} is formed with the post-recalibration noise and c_f is the constant of Lemma 11. The third term decays exponentially in T , and the whole perturbation is at most $\frac{8}{\mu}M^2B^2$ at every T . Since $\min(B^2, c_f^2/T)$ and $B^2/(1 + T/T_0)$ with $T_0 := c_f^2/B^2$ agree up to a factor of two, this is the form stated in Section 4.3.

Proof. The proof has two parts. We first verify that the fused estimate satisfies the hypotheses of Lemma 9, now with a smaller variance and a perturbation that shrinks over time, and we then bound the resulting perturbation sum, which is where the contraction of the bias comes from.

Part 1, the hypotheses. Work on the success event of Lemma 11, which has probability at least $1 - \delta$. On this event the recalibrated scores have systematic error at most $\min(B, c_f\sqrt{\log(2KT/\delta)/t})$ at step t , since recalibration maps into the score range, so the error never exceeds B , and the sharper bound of Lemma 11 applies once enough anchor pairs have accumulated. Applying Lemma 10 to the recalibrated scores decomposes the world model estimate as $u_{WM} + d_{WM}$ with $\text{Var}(u_{WM}) \leq V_{WM}$ and $\|d_{WM}\| \leq 2M \min(B, c_f\sqrt{\log(2KT/\delta)/t})$, while the anchor estimate has the same mean $(1 - \frac{1}{n})\nabla J$ with variance V_E , and the two are independent as they are computed on disjoint groups. By Lemma 12, fusing them with inverse-variance weights attains the harmonic variance, and since the weights sum to one the fused estimate keeps the common mean, so Lemma 9 applies with $\kappa_t = 1 - \frac{1}{n}$, $V = (1/V_E + 1/V_{WM})^{-1}$ and $D_t = 2M \min(B, c_f\sqrt{\log(2KT/\delta)/t})$, which already produces the geometric term and the variance term of the claim.

Part 2, the perturbation sum. What remains is the middle term of Lemma 9,

$$2\gamma \sum_{t < T} \left(1 - \frac{\gamma\mu}{4}\right)^{T-1-t} D_t^2, \quad D_t^2 \leq 4M^2 \min\left(B^2, c_f^2 \frac{\log(2KT/\delta)}{t}\right).$$

The two factors decay in opposite directions, the geometric weight is small for early t and the summand D_t^2 is small for late t , so we split the sum at $T/2$ and use the stronger effect on each half. For $t \geq T/2$ the summand is small, $D_t^2 \leq 8M^2c_f^2 \log(2KT/\delta)/T$, and the geometric weights sum to at most $\frac{4}{\gamma\mu}$, so this

half contributes at most $\frac{64c_f^2M^2}{\mu} \log(2KT/\delta)/T$, the contracting bias term. For $t < T/2$ the weight is small, $(1 - \frac{\gamma\mu}{4})^{T-1-t} \leq (1 - \frac{\gamma\mu}{4})^{T/2}$, and with the crude bound $D_t^2 \leq 4M^2B^2$ over at most $T/2$ terms this half contributes at most $4\gamma TM^2B^2(1 - \frac{\gamma\mu}{4})^{T/2}$, which vanishes exponentially in T . Finally, bounding the same sum with the constant $D \equiv 2MB$ instead gives $\frac{8}{\mu}M^2B^2$, so the perturbation never exceeds the floor of Theorem 3 and decays as $\tilde{O}(M^2c_f^2/T)$, which is the contracting form stated in Section 4.3. $\square$

Corollary 7. *Assume in addition that the recalibration map is non-expansive, so that the post-recalibration noise satisfies $\eta^2 \leq \sigma^2$. Then, comparing Theorem 4 with Theorem 3 term by term, the leading terms coincide, there exists a finite T_1 such that for all $T \geq T_1$ the bias term of WMRL is strictly smaller than the floor $\frac{32M^2}{\mu}B^2$ and converges to zero as $T \rightarrow \infty$, and the variance term of WMRL is smaller by the factor $1 + V_{WM}/V_E > 1$.*

Proof. The leading terms are identical by construction. For the bias, the term of Theorem 4 falls below $\frac{32M^2}{\mu}B^2$ as soon as $\log(2KT/\delta)/T \leq B^2/(2c_f^2)$ and the exponentially decaying remainder is below the same threshold, both of which hold for all T beyond some finite T_1 , and the term itself converges to zero. For the variance, Lemma 12 writes the fused variance as $V_{WM}/(1 + V_{WM}/V_E)$, and non-expansiveness gives $\eta^2 \leq \sigma^2$, so the V_{WM} of Theorem 4, formed with the post-recalibration noise η , is no larger than the V_{WM} of Theorem 3, formed with σ . $\square$

Remark 8 (Plug-in weights). *The bounds above are stated with the exact variances, while training uses the plug-in ratio of Section 3.3, whose varying part is measured online and whose scale is fixed once by the normalization $c := 1/\hat{\eta}_{\text{cal}}^2$. A misspecified scale would generically cost a first-order term. It does not here, because the inverse-variance weights sit at the minimum of the fused variance, where the first-order term vanishes identically. Writing $\rho := V_{WM}/V_E$ for the true ratio, $\hat{\rho}$ for the one used and $\varepsilon_\rho := (\hat{\rho} - \rho)/\rho$ for its relative error, the last claim of Lemma 12 with $\alpha = \rho/(1 + \rho)$ becomes*

$$\frac{\text{Var}(\hat{g}_{\hat{\rho}})}{\text{Var}(\hat{g}_\rho)} = 1 + \frac{\rho}{(1 + \rho)^2} \varepsilon_\rho^2 + O(\varepsilon_\rho^3), \quad \frac{\rho}{(1 + \rho)^2} \leq \frac{1}{4},$$

so a relative error in the ratio inflates the variance by at most $\varepsilon_\rho^2/4$ to this order. The scales measured in our runs (Appendix D) put this excess below three percent at the calibration point, so the variance term of Theorem 4 is essentially unaffected.

B.4. Auxiliary lemmas

This subsection collects the four lemmas used above. Before each statement we recall why the lemma is needed and where it is consumed.

The first lemma is the engine behind both theorems. It answers how far gradient ascent can still progress when the estimate carries a bounded perturbation and extra variance, and every convergence statement of this paper is an instance of it, Proposition 5 with $D_t \equiv 0$, Theorem 3 with $(V, D) = (V_{WM}, 2MB)$, and Theorem 4 with the harmonic variance and a shrinking D_t . The middle term of its bound is the weighted telescoping sum that the two theorems then control in different ways.

Lemma 9 (Perturbed ascent under gradient domination). *Let $\theta_{t+1} = \theta_t + \gamma\hat{g}_t$ with $\gamma \leq 1/(8L)$. Suppose that, conditioned on θ_t , the estimate decomposes as $\hat{g}_t = u_t + d_t$, where $\mathbb{E}[u_t | \theta_t] = \kappa_t \nabla J(\theta_t)$ with $\kappa_t \in [\frac{1}{2}, 1]$,*

$\text{Var}(u_t \mid \theta_t) \leq V$, and $\|d_t\| \leq D_t$ almost surely. Then

$$J^* - \mathbb{E}[J(\theta_T)] \leq \left(1 - \frac{\gamma\mu}{4}\right)^T \Delta_0 + 2\gamma \sum_{t=0}^{T-1} \left(1 - \frac{\gamma\mu}{4}\right)^{T-1-t} D_t^2 + \frac{4L\gamma}{\mu} V.$$

In particular, if $D_t \equiv D$, the middle term is at most $\frac{8}{\mu} D^2$.

Proof. The plan mirrors the three steps of Proposition 5. We lower-bound the useful progress $\langle \nabla, \hat{g}_t \rangle$, upper-bound the harmful second moment $\|\hat{g}_t\|^2$, and combine the two through smoothness into a recursion for δ_t . Write $\nabla := \nabla J(\theta_t)$ and let $\mathbb{E}_t$ denote expectation conditioned on θ_t ; under $\mathbb{E}_t$ the quantities $J(\theta_t)$ and ∇ are fixed, so only the terms containing $\hat{g}_t$ are affected. By L -smoothness, $J(\theta_{t+1}) \geq J(\theta_t) + \gamma \langle \nabla, \hat{g}_t \rangle - \frac{L\gamma^2}{2} \|\hat{g}_t\|^2$.

Step 1, the progress term. Splitting $\hat{g}_t = u_t + d_t$, the inner product separates into signal and contamination, $\mathbb{E}_t \langle \nabla, \hat{g}_t \rangle = \kappa_t \|\nabla\|^2 + \langle \nabla, \mathbb{E}_t d_t \rangle$. The contamination can only be bounded by Cauchy–Schwarz, $\langle \nabla, \mathbb{E}_t d_t \rangle \geq -\|\nabla\| D_t$, and a product of two different quantities is inconvenient in a recursion, so we separate it with Young’s inequality $ab \leq a^2/4 + b^2$,

$$\mathbb{E}_t \langle \nabla, \hat{g}_t \rangle \geq \kappa_t \|\nabla\|^2 - \|\nabla\| D_t \geq (\kappa_t - \tfrac{1}{4}) \|\nabla\|^2 - D_t^2.$$

The choice of $\frac{1}{4}$ sacrifices a quarter of the signal to turn the cross term into a pure D_t^2 .

Step 2, the second moment. We control mean and fluctuation separately. The mean obeys $\|\mathbb{E}_t \hat{g}_t\| \leq \kappa_t \|\nabla\| + D_t$, so $(a+b)^2 \leq 2a^2 + 2b^2$ gives $\|\mathbb{E}_t \hat{g}_t\|^2 \leq 2\|\nabla\|^2 + 2D_t^2$, and the fluctuation obeys $\text{Var}(\hat{g}_t) \leq 2\text{Var}(u_t) + 2\text{Var}(d_t) \leq 2V + 2D_t^2$. Adding the two, $\mathbb{E}_t \|\hat{g}_t\|^2 \leq 2\|\nabla\|^2 + 4D_t^2 + 2V$.

Step 3, combine and unroll. Substituting the two bounds into the smoothness inequality, and simplifying with $\kappa_t \geq \frac{1}{2}$ and $L\gamma \leq \frac{1}{8}$,

$$\begin{aligned} \mathbb{E}_t J(\theta_{t+1}) &\geq J(\theta_t) + \gamma(\kappa_t - \tfrac{1}{4} - L\gamma) \|\nabla\|^2 - \gamma D_t^2 (1 + 2L\gamma) - L\gamma^2 V \\ &\geq J(\theta_t) + \tfrac{\gamma}{8} \|\nabla\|^2 - 2\gamma D_t^2 - L\gamma^2 V. \end{aligned}$$

Gradient domination replaces the gradient by the gap, $\|\nabla\|^2 \geq 2\mu (J^* - J(\theta_t))$, and taking total expectations through the tower property turns the display into a recursion on the deterministic sequence $\delta_t := J^* - \mathbb{E}[J(\theta_t)]$,

$$\delta_{t+1} \leq \left(1 - \frac{\gamma\mu}{4}\right) \delta_t + 2\gamma D_t^2 + L\gamma^2 V.$$

Multiplying the step- t inequality by $(1 - \frac{\gamma\mu}{4})^{T-1-t}$ and summing over $t = 0, \dots, T-1$, the intermediate δ_t cancel telescopically and the claim follows. For constant D the geometric sum $\sum_{k \geq 0} (1 - \frac{\gamma\mu}{4})^k = \frac{4}{\gamma\mu}$ bounds the middle term by $\frac{8}{\mu} D^2$ and the last term by $\frac{4L\gamma}{\mu} V$. $\square$

The second lemma is what allows Lemma 9 to be fed with world model rewards. It converts the reward-level error model of Equation (3) into exactly the three quantities that Lemma 9 consumes, the mean of the clean part, its variance, and the almost-sure size of the perturbation, and it also produces the variance relation $V_{WM} = (1 + c\sigma^2)V_E$ used in Section 3.3.

Lemma 10 (From reward error to gradient error). *Let $\hat{g}_k$ be computed with true scores and $\hat{g}_k^{WM}$ with predicted scores. Under Assumptions 2 and 6, $\hat{g}_k^{WM} = \hat{g}_k + g_b + g_{\xi}$, where*

$$(i) \quad \mathbb{E}[\hat{g}_k] = (1 - \tfrac{1}{n}) \nabla J(\theta) \text{ and } V_E = \text{Var}(\hat{g}_k) \leq M^2;$$

- (ii) $g_b = \frac{1}{n} \sum_i (b_i - \bar{b}) s_i$ satisfies $\|g_b\| \leq 2MB$ almost surely, and vanishes whenever b is constant within the group;
- (iii) g_ξ has zero conditional mean, is uncorrelated with $\hat{g}_k$, and satisfies $\mathbb{E}\|g_\xi\|^2 \leq 4M^2\sigma^2/n$, so that $V_{WM} = V_E + 4M^2\sigma^2/n = (1 + c\sigma^2) V_E$ with $c = 4M^2/(nV_E)$, the relation used in Section 3.3.

Proof. Recall from Equation (5) that $\hat{g}_k = \frac{1}{n} \sum_i A_i s_i$, where the advantage $A_i = r_i - \frac{1}{n} \sum_j r_j$ is a linear function of the scores of its group. Abbreviate $r_i := r(\tau_i)$, $b_i := b(\tau_i)$, and $\xi_i := \xi(\tau_i)$.

The decomposition is pure algebra. Replacing every true score by the predicted one substitutes into the same linear formula, so with $\hat{r}_i = r_i + b_i + \xi_i$ the new advantage is

$$\hat{A}_i = \hat{r}_i - \frac{1}{n} \sum_j \hat{r}_j = A_i + (b_i - \bar{b}) + (\xi_i - \bar{\xi}),$$

with $\bar{b}$, $\bar{\xi}$ the group means. Multiplying by s_i/n and summing over i gives $\hat{g}_k^{WM} = \hat{g}_k + g_b + g_\xi$ with $g_b := \frac{1}{n} \sum_i (b_i - \bar{b}) s_i$ and $g_\xi := \frac{1}{n} \sum_i (\xi_i - \bar{\xi}) s_i$; no probabilistic argument is involved, the three parts simply collect the score, bias, and noise contributions. It remains to control each part in the form that Lemma 9 consumes, the mean and variance of the clean part, the almost-sure size of the bias part, and the second moment of the noise part.

(i) *The clean part.* Expanding the advantage, $\mathbb{E}[A_i s_i] = \mathbb{E}[r_i s_i] - \frac{1}{n} \sum_j \mathbb{E}[r_j s_i]$. For $j \neq i$ the two trajectories are independent given the task, so the expectation factorizes, and $\mathbb{E}[s_i] = \mathbb{E}[\nabla \log \pi_\theta] = 0$ kills the term; the $j = i$ term contributes $\frac{1}{n} \mathbb{E}[r_i s_i]$. Summing over i , $\mathbb{E}[\hat{g}_k] = (1 - \frac{1}{n}) \mathbb{E}[r(\tau) \nabla \log \pi_\theta(\tau)] = (1 - \frac{1}{n}) \nabla J(\theta)$, where the last step is the policy gradient identity. For the variance, $r_i \in [0, 1]$ forces $|A_i| \leq 1$, hence $\|\hat{g}_k\| \leq \frac{1}{n} \sum_i \|s_i\| \leq M$, and a vector bounded by M has variance at most $\mathbb{E}\|\hat{g}_k\|^2 \leq M^2$, so $V_E \leq M^2$.

(ii) *The bias part.* It is deterministic given the trajectories, so what Lemma 9 needs is an almost-sure bound. Since $|b_i| \leq B$, the centered value obeys $|b_i - \bar{b}| \leq 2B$, and therefore $\|g_b\| \leq \frac{1}{n} \sum_i |b_i - \bar{b}| \|s_i\| \leq 2MB$. If b is constant within the group, the centering removes it entirely, $b_i - \bar{b} = 0$, which is why only the within-group variation of the bias ever reaches the gradient.

(iii) *The noise part.* It should act as extra variance, so we verify that it is conditionally zero-mean and compute its second moment. Rewrite $g_\xi = \frac{1}{n} \sum_i \xi_i (s_i - \bar{s})$ with $\bar{s} := \frac{1}{n} \sum_j s_j$. Conditioned on the trajectories the ξ_i are independent and zero-mean, so $\mathbb{E}[g_\xi | \tau_{1:n}] = 0$ and all cross terms of $\mathbb{E}\|g_\xi\|^2$ vanish, leaving

$$\mathbb{E}\|g_\xi\|^2 = \frac{1}{n^2} \sum_i \mathbb{E}[\mathbb{E}[\xi_i^2 | \tau] \|s_i - \bar{s}\|^2] \leq \frac{\sigma^2}{n^2} \cdot n \cdot (2M)^2 = \frac{4M^2\sigma^2}{n}.$$

The same conditional zero mean makes g_ξ uncorrelated with $\hat{g}_k$, so the two variances add, $V_{WM} = V_E + 4M^2\sigma^2/n = (1 + c\sigma^2) V_E$ with $c = 4M^2/(nV_E)$, the stated relation. $\square$

The third lemma supplies the shrinking perturbation bound D_t used in the proof of Theorem 4. It quantifies how fast the isotonic recalibration of Equation (4) learns the distortion ϕ from the accumulating anchor pairs, and its $1/\sqrt{t}$ rate is what ultimately becomes the contraction of the bias term.

Lemma 11 (Recalibration error). *Under Assumption 6, with probability at least $1 - \delta$ simultaneously for all $1 \leq t \leq T$, the recalibration map $\hat{f}_t$ is well defined once enough anchor pairs have accumulated, and the systematic error of the recalibrated score obeys*

$$|\mathbb{E}[\hat{f}_t(\hat{r}) | \tau] - r(\tau)| \leq c_f \sqrt{\log(2KT/\delta)/t},$$

where K is the number of distinct scores the benchmark produces and the constant c_f is given in Equation (6) of the proof.

Proof. We first fix notation for the scores our benchmarks produce. The leaderboard percentile is computed against a finite leaderboard and the VLA reward is binary, so the true scores take finitely many values $\ell_1 < \dots < \ell_K$, and the recalibration of Equation (4) fits one vertex per level. Write $\Delta_\ell := \max_k(\ell_{k+1} - \ell_k)$ for the largest gap between adjacent levels, $\Delta_\phi := \min_k(\phi(\ell_{k+1}) - \phi(\ell_k))$ for the smallest separation the distortion leaves between them, q for the smallest fraction of the anchor pairs that any level receives, and a for the number of new pairs per step. The rate below is informative when $\Delta_\phi > 0$ and $q > 0$, that is, when the distortion collapses no two levels onto one another and no level is starved of anchor pairs, which is the discrete-level analogue of the arm gaps of bandit analysis [63] and of the coverage conditions of off-policy evaluation [64], and is what turns isotonic regression from a worst-case problem into one with a parametric rate [65]. In terms of these quantities the constant of the statement is

$$c_f := \frac{2\Delta_\ell}{\Delta_\phi} \sqrt{\frac{1}{2qa}}. \quad (6)$$

The proof then has three stages that mirror the recalibration pipeline. We first show that the empirical mean of the predictions at each score level concentrates, then that the isotonic projection can only inherit this accuracy, and finally that the interpolated map turns accurate vertices into accurate calibrated scores.

Stage 1, level means concentrate. Coverage is what makes this stage possible, as a level that received no pairs could never be calibrated. At step t each level holds at least qat pairs, each of the form $\hat{r} = \phi(\ell_k) + \zeta$ with ζ zero-mean and $|\zeta| \leq 1$, so Hoeffding’s inequality gives $\Pr(|\hat{\mu}_k - \phi(\ell_k)| \geq s) \leq 2e^{-2qats^2}$. The guarantee is needed simultaneously for every level and every step, so we take a union bound over the K levels and T steps, and the choice $s = e(t) := \sqrt{\log(2KT/\delta)/(2qat)}$ keeps the total failure probability below δ .

Stage 2, the projection does not hurt. The isotonic fit replaces the raw means $\hat{\mu}_k$ by the closest monotone sequence $\hat{\phi}_k$, and we must check that this projection cannot push an estimate away from the truth. The min–max representation $\hat{\phi}_k = \min_{v \geq k} \max_{u \leq k} \text{avg}(\hat{\mu}_u, \dots, \hat{\mu}_v)$ expresses every fitted value as an average of raw means, each within $e(t)$ of its true value. For any $v \geq k$, monotonicity of ϕ gives $\max_{u \leq k} \text{avg}[u, v] \geq \text{avg}[k, v] \geq \text{avg}(\phi(\ell_k), \dots, \phi(\ell_v)) - e(t) \geq \phi(\ell_k) - e(t)$, and taking the minimum over v preserves the bound; choosing $v = k$ gives symmetrically $\hat{\phi}_k \leq \phi(\ell_k) + e(t)$. So the fitted vertices are as accurate as the raw means.

Stage 3, from vertices to calibrated scores. Separation is what makes this stage possible, as levels the distortion has collapsed onto one another could not be told apart by any monotone fit. When $e(t) \leq \Delta_\phi/4$, consecutive fitted vertices stay separated, $\hat{\phi}_{k+1} - \hat{\phi}_k \geq \Delta_\phi - 2e(t) \geq \Delta_\phi/2$, so the piecewise-linear interpolation $\hat{f}_t$ is well defined and its Lipschitz constant is at most $\Delta_\ell/(\Delta_\phi/2) = 2\Delta_\ell/\Delta_\phi$. A trajectory with true score ℓ_k produces predictions with conditional mean $\phi(\ell_k)$, and $\hat{f}_t$ maps the fitted vertex $\hat{\phi}_k$ exactly to ℓ_k , so the systematic error of the calibrated score is at most the Lipschitz constant times the vertex error, $|\hat{f}_t(\phi(\ell_k)) - \ell_k| = |\hat{f}_t(\phi(\ell_k)) - \hat{f}_t(\hat{\phi}_k)| \leq \frac{2\Delta_\ell}{\Delta_\phi} e(t)$, which is the claim. $\square$

The last lemma justifies the fusion rule of Equation (5) and supplies the variance V used in the proof of Theorem 4. Its final claim, that the fused variance is flat around the optimal weight, is what Remark 8 invokes to argue robustness to the estimated ratio.

Lemma 12 (Optimal linear fusion). *Let g_1 and g_2 be independent unbiased estimates of the same quantity with variances V_1 and V_2 . Among all their unbiased linear combinations, that is, with weights summing to one,*

the variance is minimized by weights proportional to V_1^{-1} and V_2^{-1} , and the minimal value is

$$\left(\frac{1}{V_1} + \frac{1}{V_2}\right)^{-1} < \min(V_1, V_2).$$

Moreover, for any weight α on g_1 , $\text{Var}(\hat{g}_\alpha) = \text{Var}(\hat{g}_{\alpha^*}) + (V_1 + V_2)(\alpha - \alpha^*)^2$, so a misspecified weight costs only a second-order term.

Proof. Write the combination with a single weight, $\hat{g}_\alpha = \alpha g_1 + (1 - \alpha)g_2$, which is unbiased for every α , so the choices differ only in variance. By independence, $\text{Var}(\hat{g}_\alpha) = \alpha^2 V_1 + (1 - \alpha)^2 V_2$, a strictly convex quadratic in α , so the unique minimizer is found by setting the derivative to zero, $\alpha^* = V_2 / (V_1 + V_2)$, that is, weights proportional to the inverse variances. Substituting α^* back gives $V_1 V_2 / (V_1 + V_2)$, which equals the stated harmonic form and is strictly smaller than each of V_1 and V_2 because both precisions are positive. Finally, expanding the quadratic around its minimizer gives $\text{Var}(\hat{g}_\alpha) = \text{Var}(\hat{g}_{\alpha^*}) + (V_1 + V_2)(\alpha - \alpha^*)^2$, since its second derivative is $2(V_1 + V_2)$; this is the second-order insensitivity invoked in Remark 8. $\square$

Remark 13 (On restricting to linear combinations). *Lemma 12 optimizes within the linear family, which is the weakest requirement that suits our setting, as it uses only the second moments of the two estimates and preserves unbiasedness whatever their distribution. If the estimates are in addition taken to be Gaussian, which group averages approach as the group size grows, the same inverse-variance weights are minimum-variance among all unbiased estimators, so the linear restriction is not what limits the result.*

C. Choice of the Task Pool

Section 5.1 builds the task pool from MLE-Dojo rather than from MLE-Bench directly, for three reasons.

First, part of the original pool is no longer usable. Two of the underlying Kaggle competitions, `detecting-insults-in-social-commentary` and `the-icml-2013-whale-challenge-right-whale-redux`, have been permanently closed, and their official data downloads and submission entries are gone.

Second, the medal-based metric fails to measure model ability. MLE-Bench scores an agent by the number of gold, silver, and bronze medals. At the model scales we study, an agent earns medals on only one or two fixed competitions, so the medal count stays flat no matter how much the agent improves on the rest of the pool. The counts are also skewed, with more golds than silvers or bronzes in published evaluations [31], because a handful of tasks admit near-perfect accuracy and hand out gold medals while the remaining tasks hand out nothing. The ranking against human competitors is what reflects ability, so our protocol reports the leaderboard percentile, which credits improvement on every task.

Third, contamination. The competitions and many of their winning solutions predate the pretraining corpora of current models, a risk acknowledged both in the original release [29] and in the accompanying OpenAI blog post [66], and the public evaluation has not been actively maintained since.

MLE-Dojo wraps a superset of the same competitions in an interactive environment with live grading, which avoids the dead entries, exposes leaderboard percentile directly, and allows a free re-partition of the pool. Manually splitting a public pool into train and test portions follows common practice, and our split is fixed once and shared by all configurations.

Why these training and test sets. The partition follows one deterministic rule, fixed before any training, and is designed to keep the categories balanced on both sides. Within each of the three categories (tabular, text,

image), the 20 smallest competitions by dataset size are selected so that every task fits the execution sandbox, the 15 smallest of them form the training set, and the remaining 5 form the held-out set. Audio competitions are excluded because they do not fit the sandbox, and one held-out task (billion-word-imputation) is dropped because its grader depends on a package unavailable in our environment, leaving 45 training and 14 held-out competitions. The rule has a useful side effect, as every held-out task is larger than every training task of its category, so the held-out evaluation also measures extrapolation from small training tasks to larger unseen ones. We also verified that no task appears on both sides.³ The data of every competition was cached locally in advance, so training and grading never depend on the Kaggle servers. Table 5 lists the training competitions and Table 6 the held-out ones.

Table 5: The 45 training competitions of MLE-Dojo (train), by category.

Tabular	Text	Image
mercedes-benz-greener-manufacturing	kaggle-llm-science-exam	aerial-cactus-identification
icr-identify-age-related-conditions	movie-review-sentiment-analysis-kernels-only	leaf-classification
kobe-bryant-shot-selection	llm-detect-ai-generated-text	denoising-dirty-documents
home-data-for-ml-course	random-acts-of-pizza	facial-keypoints-detection
spooky-author-identification	chaii-hindi-and-tamil-question-answering	statoil-iceberg-classifier-challenge
liberty-mutual-group-property-inspection-prediction	nbme-score-clinical-patient-notes	tgs-salt-identification-challenge
detecting-insults-in-social-commentary	20-newsgroups-ciphertext-challenge	global-wheat-detection
walmart-recruiting-store-sales-forecasting	word2vec-nlp-tutorial	whale-categorization-playground
prudential-life-insurance-assessment	jigsaw-toxic-comment-classification-challenge	dog-breed-identification
unimelb	text-normalization-challenge-english-language	plant-pathology-2020-fgvc7
nomad2018-predict-transparent-conductors	wsdm-cup-multilingual-chatbot-arena	dogs-vs-cats-redux-kernels-edition
amazon-employee-access-challenge	text-normalization-challenge-russian-language	petfinder-pawpularity-score
poker-rule-induction	llm-classification-finetuning	plant-seedlings-classification
allstate-purchase-prediction-challenge	stumbleupon	shopee-product-matching
dont-call-me-turkey	linking-writing-processes-to-writing-quality	the-nature-conservancy-fisheries-monitoring

³One held-out task shares the Jigsaw competition family with a training task, with different data and a different metric.

Table 6: The 14 held-out competitions of MLE-Dojo (test).

Competition	Category	Task
GiveMeSomeCredit	Tabular	Predict two-year default risk of borrowers
forest-cover-type-kernels-only	Tabular	Predict forest cover type from cartographic features
novozymes-enzyme-stability-prediction	Tabular	Rank thermostability of enzyme variants
integer-sequence-learning	Tabular	Predict the next term of integer sequences
afsis-soil-properties	Tabular	Predict five soil properties from infrared spectra
quora-question-pairs	Text	Decide whether two questions are duplicates
AI4Code	Text	Recover the order of markdown cells in notebooks
jigsaw-unintended-bias-in-toxicity-classification	Text	Detect toxicity while controlling identity bias
quora-insincere-questions-classification	Text	Flag insincere questions
uw-madison-gi-tract-image-segmentation	Image	Segment stomach and bowel in MRI scans
invasive-species-monitoring	Image	Detect an invasive plant species in photos
kuzushiji-recognition	Image	Detect and transcribe cursive Japanese characters
bengaliai-cv19	Image	Classify components of handwritten Bengali graphemes
cassava-leaf-disease-classification	Image	Classify cassava leaf diseases

DSBench. The transfer suite is the data-modeling split of DSBench [30], whose tasks are all drawn from Kaggle (74 tasks per the paper, 75 task directories in the released data). We keep the 60 tasks that run end to end in our sandbox. The other 15 are excluded because the training data is too large for the sandbox (seven tasks), the sample submission is missing (eight tasks), or the submission format exceeds the scaffold’s column limit (one task). The category columns of Table 1 contain 19 binary classification, 10 multi-class classification, and 23 regression tasks, and the remaining 8 tasks with non-standard metrics form the Other column. We also checked these 60 tasks against the 45 training competitions with exact, normalized, and fuzzy name matching, and found no overlap.

D. Experimental Details

Training configuration. Both scales train with GRPO, eight groups per step and $n = 8$ trajectories per group, with up to four interaction turns per trajectory, at most 4096 generated tokens per turn, and observations truncated to 1024 tokens. Optimization uses a constant learning rate of 1×10^{-6} , a KL coefficient of 0.04, and an entropy coefficient of 0.002, and rollouts are sampled at temperature 1.0 with top- p 1.0. Real execution

runs each solution in an isolated environment with a 1200-second budget. The world model serves the agent backbone with a 12k-token context and returns its prediction within a 1024-token budget, and every step at least one group is graded by real execution to feed the anchor pool.

Sandbox environment. Each solution executes in an isolated sandbox built as a Python 3.11 virtual environment on Linux, with a single ~ 40 GB NVIDIA GPU and the path contract of the agent prompt, reading inputs from DATA_DIR and writing the submission to SUBMISSION_PATH. The environment holds 122 packages including transitive dependencies, and installing the directly requested set of Table 7 into a clean virtual environment reproduces it.

Table 7: **Libraries preinstalled in the execution sandbox**, by category, with the exact versions.

Category	Libraries (version)
Scientific computing	numpy 1.26.4, pandas 2.2.3, scipy 1.17.1, statsmodels 0.14.6, sympy 1.14.0, networkx 3.6.1
Classical ML	scikit-learn 1.4.2, xgboost 3.2.0, lightgbm 4.6.0, catboost 1.2.10, joblib 1.5.3
Deep learning (GPU)	torch 2.12.0 (CUDA 13.0), torchvision 0.27.0, tensorflow 2.21.0, keras 3.14.1
NLP	transformers 5.12.1, tokenizers 0.22.2, nltk 3.9.4
Imaging and plotting	Pillow 12.2.0, matplotlib 3.10.9, seaborn 0.13.2, plotly 6.6.0
Utilities	h5py 3.14.0, tqdm 4.67.3, requests 2.34.2, regex

Prompting the world model. The world model is never fine-tuned and works by prompting alone. Its prompt contains the same task description that the agent sees and the agent’s current solution, and it is instructed to simulate the execution and report the outcome in the same format as the real environment, from which the score is aggregated. The context is capped at 12k tokens and the prediction at 1024 tokens. Appendix E sketches the structure of this prompt and of the agent prompt.

Recalibration. The monotone map of Equation (4) is the identity until 200 anchor pairs have accumulated, is first fit at that point, and is refit after every 64 new pairs so that the correction follows the drift of the world model.

Calibrating the reference disagreement. The disagreement $\hat{\eta}^2$ is normalized by the within-group spread of the true scores, and this normalization is what lets a reward-level statistic estimate a gradient-level ratio. The sampling variance V_E and the noise-injected excess of V_{WM} arise from the same estimator applied to the same trajectories, so the conversion from score fluctuation to gradient fluctuation largely cancels in their ratio, and what remains is the noise variance measured against the natural spread of the scores, which is $\hat{\eta}^2$ up to a conversion factor of order one shared between the two streams. A single measurement at the end of warmup fixes this factor, $c = 1/\hat{\eta}_{\text{cal}}^2$, and we find the calibrated value in one of our actual runs to be 0.96, confirming that it is indeed of order one. The fused variance is additionally second-order insensitive to the weight (Lemma 12), so the precision of this single measurement is not critical.

Bounded anchor weight. For training stability, we bound the anchor factor of Equation (5) within $[1, w_{\max}]$ with $w_{\max} = 4$, which acts as a regularization on the fused gradient. The lower bound keeps anchor groups

from being down-weighted below the predicted groups, and the upper bound prevents a transient spike of the measured disagreement from letting a few anchor groups dominate a batch. The tracked noise is likewise estimated over a recent window of the pool $\mathcal{P}$ rather than the full history, so that the measurement tracks the current policy.

VLA setup. The policy is MiniVLA-1B [58], a Prismatic-style model with a Qwen2.5-0.5B language backbone and a vector-quantized action head that encodes an eight-step action chunk into seven discrete tokens, pretrained on LIBERO-90. All models first run supervised fine-tuning on the 50 official demonstrations per task, and then GRPO for 40 steps. Each step samples 64 rollouts per task, and each rollout takes at most 520 environment steps, the benchmark’s standard horizon. Training uses the first 16 official initial states of each task while evaluation uses all 50, so most evaluated initial states are never seen in training. Evaluation is greedy, with eight repetitions per initial state, since the simulator’s physics is stochastic.

E. Prompts

Both prompts are too long to reproduce in full, so we sketch their structure below, quote the load-bearing passages verbatim, and mark every omission with [...].

The agent prompt. Each turn, the AutoResearch agent sees a fixed system prompt, a user prompt holding an automatically generated task overview, and the full history of its previous attempts with their execution feedback. The system prompt consists of the seven segments sketched below. The user prompt is the first user message of the conversation, as the later turns simply accumulate the interaction history on top of it, and the task overview inside is generated by pure introspection over the task files, with no manual per-task information.

System prompt of the AutoResearch agent

Role and objective.

You are an expert ML engineer competing to MAXIMIZE your leaderboard Position Score over UP TO K attempts (Position Score = your percentile rank, higher = better; earned ONLY when your code runs and writes a VALID submission). [...]

Two-phase strategy.

PHASE 1 -- LAND A VALID SUBMISSION FIRST (turn 1): write a SIMPLE solution you are confident RUNS end-to-end and writes SUBMISSION_PATH in the exact required format. A valid submission is your safety net. [...]

PHASE 2 -- IMPROVE INCREMENTALLY (later turns): START FROM YOUR LAST WORKING CODE, copy it, and change EXACTLY ONE thing per turn. [...]

If a change ERRORS or scores WORSE, discard it and try a DIFFERENT single change. Never rewrite from scratch once something works.

Environment contract.

ENVIRONMENT CONTRACT (these Python variables are ALREADY defined when your code runs): [...]

DATA_DIR: absolute path to the folder with the data files listed below. [...]

SUBMISSION_PATH: absolute path to write your submission CSV to. [...]

A modern NVIDIA GPU (~40 GB) is available. [...]

Do NOT hardcode '/kaggle/...', '/content/...', or any other path.

Sandbox runtime.

Runtime: Python 3.11 on Linux (this is the sandbox where YOUR code executes). Installed libraries you may import (name version): numpy 1.26, pandas 2.2, scikit-learn 1.4, xgboost 3.2, torch 2.12 (CUDA), transformers 5.12 [...]

Anything NOT in this list is not installed (e.g. no opencv, no jax).

Response format.

EACH TURN, respond in TWO parts, in this order: [...]

1) ANALYSIS (2-4 short sentences of plain text, FIRST): name exactly what went wrong and the ONE change that fixes it, or the ONE change you will make to raise the score. [...]

2) CODE: a SINGLE fenced Python block -- a complete, self-contained script.

Feedback loop.

After your code runs, the environment returns your print() output and -- if a valid submission was written -- your leaderboard Position and Raw Score; otherwise the full error traceback. [...]

Rules.

Keep a valid submission as your fallback; only replace it with code that RUNS and scores BETTER. [...]

The submission CSV must match the required format EXACTLY. Keep code efficient so it finishes within the time limit.

User prompt of the AutoResearch agent

```
Competition: <name>
=== DESCRIPTION ===
<head of the official competition description, including its evaluation metric> [...]
=== DATA INVENTORY ===
<every file with its size; per-CSV schema with row count and every column's dtype and example values; ZIP and directory contents; sampled image dimensions> [...]
=== TARGETS ===
<the columns present in the training data but absent from the test data>
=== SUBMISSION CONTRACT ===
<the exact required columns and example rows of sample_submission.csv>
Begin.
```

The world model prompt. The world model shares one simulator system prompt across eight parallel single-aspect checks, which cover imports and names, API calls against the installed versions, file and column references against the data inventory, shapes and dtypes, the compute budget, remaining runtime errors, submission validity, and solution quality. All eight checks share the same system prompt. The user prompt of each check holds its specific instruction followed by the case block, namely the task overview above and the agent's code with 1-indexed line numbers, and the check returns a verdict in strict JSON. The verdicts are then aggregated into the scalar score $\hat{r}$, where failures cap the score and a valid solution is scored by the quality check. The user prompt shown below is that of the file and column reference check.

System prompt of the world model

Role.

You are a precise execution-and-grading SIMULATOR for Kaggle/MLE Python solutions. You do NOT execute code. You predict -- by careful static analysis -- exactly what the real sandbox would report.

Environment brief.

Sandbox: Python 3.11 on Linux, ONE ~40GB GPU available. Installed (name version): numpy 1.26, pandas 2.2 [...]

Execution is killed at 1200s. The code reads data from os.environ['DATA_DIR'] and MUST write predictions to os.environ['SUBMISSION_PATH']. [...]

Inputs.

You are given the TASK OVERVIEW (with a DATA INVENTORY: real files, columns, row counts,

dtypes, and the sample_submission format) and the agent's CODE (its lines are 1-indexed as shown). Analyze ONLY the failure mode this check asks about.

Calibration policy.

CRITICAL -- DEFAULT TO PASS. Real submissions usually RUN FINE; most code does NOT trigger this check's failure mode. [...]

Return verdict='fail' ONLY when you can cite a SPECIFIC line and a CONCRETE, CERTAIN reason it raises this exact error. [...]

A false 'fail' on working code is WORSE than a missed error. When in doubt, PASS.

Error specificity.

If you do fail it, be concrete: identify the EXACT offending source line, its line number, the precise Python exception class, and the real error message. [...]

The agent will READ your env_feedback to fix its code next turn, so env_feedback MUST look byte-similar to the real sandbox output. [...]

Output format.

Output STRICT JSON only, no prose around it:

```
{"reason": "<= 2 sentences, the specific cause>", "verdict": "<pass|fail>", "confidence":  
<float 0..1>, "error_type": [...], "error_line": [...], "env_feedback": [...]}
```

User prompt of the world model

CHECK: Will this code raise FileNotFoundError or KeyError(column) -- i.e. does it reference a file/path/column that does not match the DATA INVENTORY? [...]

verdict=fail if a FileNotFoundError or KeyError is certain; env_feedback = the exact message.

=== TASK: <competition name> ===

=== TASK OVERVIEW + DATA INVENTORY ===

<the task overview above> [...]

=== AGENT CODE (lines are 1-indexed) ===

1 <line 1 of the agent's script>

2 <line 2>

[...]